

Real-Time Red Color Detection with OpenCV

1. Import Libraries

```
python
import cv2
import numpy as np
```

- **cv2**: OpenCV library used for image/video processing.
- **numpy**: Used to create numeric arrays, such as defining color ranges.

2. Open the Camera

```
python
cap = cv2.VideoCapture(0)
```

- Activates your default camera (0).
- Use 1 or 2 for an external USB camera.

3. Continuous Frame Loop

```
python
while True:
```

- Keeps capturing frames from the camera in a loop until you quit.

4. Read Frames

```
python
ret, frame = cap.read()
```

- `ret` returns True if frame captured successfully.
- `frame` contains the actual video frame.

5. Safety Check

```
python
if not ret:
    break
```

- If frame capture fails, loop breaks safely.

6. Convert Frame to HSV

```
python
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
```

- Converts color from BGR (default in OpenCV) to HSV.
- HSV separates brightness and color tone — easier for color detection.

7. Define Red Color Range

python

```
lower_red = np.array([86, 120, 70])
```

```
upper_red = np.array([125, 255, 255])
```

- Defines the minimum and maximum HSV values that represent “red.”
- Any pixel between these two values will be considered part of the red color.

8. Create a Mask

python

```
mask = cv2.inRange(hsv, lower_red, upper_red)
```

- Creates a black & white image:
 - White (255) = red area.
 - Black (0) = non-red areas ignored.

9. Apply the Mask

python

```
red_detected = cv2.bitwise_and(frame, frame, mask=mask)
```

- Shows only the red-colored objects in the frame, hiding all others.
- Makes red parts visible while turning everything else black.

10. Display Windows

python

```
cv2.imshow("Live Camera", frame)
```

```
cv2.imshow("Red Detected", red_detected)
```

- Live Camera: Original video feed.
- Red Detected: Highlights only red regions.

11. Exit When ‘q’ is Pressed

python

```
if cv2.waitKey(1) & 0xFF == ord('q'):  
    break
```

- Waits 1 millisecond for key press.
- If you press ‘q’, the loop stops and the program closes.

12. Clean Exit

python

```
cap.release()
```

```
cv2.destroyAllWindows()
```

- Releases camera so it can be used by other apps.
- Closes all OpenCV windows properly.

Summary Table

Step	Action	Purpose
1	Import libraries	To use OpenCV and NumPy
2	Open webcam	Start capturing frames
3	Convert to HSV	Better for color detection
4	Define red range	Identify red pixels
5	Create mask	Separate red from rest
6	Apply mask	Show only red areas
7	Display windows	See live and processed output
8	Quit safely	Press 'q' to stop

Full Python Code

For completeness, here's the entire code in one block, which you can include in your PDF:

```
python
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    lower_red = np.array([86, 120, 70])
    upper_red = np.array([125, 255, 255])

    mask = cv2.inRange(hsv, lower_red, upper_red)
    red_detected = cv2.bitwise_and(frame, frame, mask=mask)

    cv2.imshow("Live Camera", frame)
    cv2.imshow("Red Detected", red_detected)
```

```
        if cv2.waitKey(1) & 0xFF == ord('q'):  
            break  
  
cap.release()  
cv2.destroyAllWindows()
```